

Тема 9. Софтуерни метрики.

Метрики на качеството на програмното осигуряване

дисциплина: Технология на софтуерното производство
лектор: доц. В.Божикова

Съдържание:

- Въведение
- Измерването в софтуерното производство
- Класификация на софтуерните метрики
- Примери за софтуерни метрики
- Методологични проблеми на използването на софтуерните метрики

ОСНОВНИ ПОНЯТИЯ

Измерване в софтуерното производство

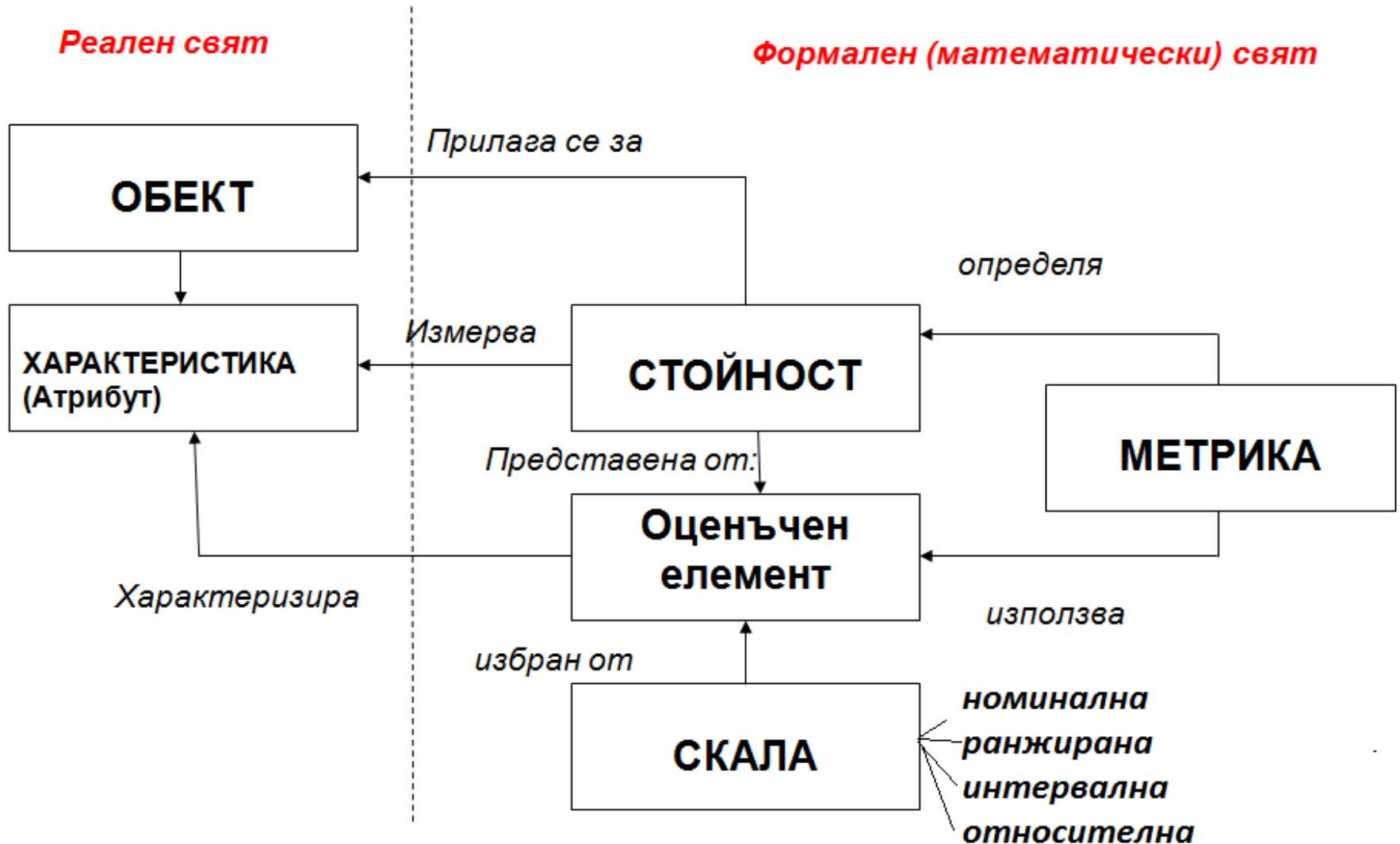
- **Измерване** – това е процес, при който, в съответствие с определени правила, на дадена характеристика на изследвания обект се съпоставят **стойности (число, но не непременно, може да е възможна степен в рамките на скала)**.
- Ако стойността е **число**, то измерването е **количествено**.
- **Мярка (measure)** е **стойността**, получена при измерване на дадена характеристика на обекта на измерване (т.е мярката - това е резултатът от измерването).

- **Метрика (metric)**, както бе посочено в предната тема, има 2 значения:
 - Другото име на мярката.
 - Процедура (подход) за намиране на мярката.

За прилагането на метрики в софтуерното производство

- **Софтуерни метрики** се прилагат за да се измерят характеристики на **обекта на софтуерното производство**, който може да е:
 - Продукт (програма, проект, спецификация, документация и др.);
 - Процес;
 - Ресурс.

Структурен модел на софтуерното измерване



- Софтуерното измерване се илюстрира с показания структурен модел.
- Характеристиките са свойства на реален софтуерен обект (н.р програма). Такива характеристики са например: размер, структура, правилност и др.
- Всяка характеристика се измерва чрез стойностите (числови или не-числови) на множество оценъчни елементи, например характеристика **“размер”** може да се измерва чрез оценъчни елементи:

- **Брой на редовете** в първичния код

- **Брой на лексемите** в първичния код

Лексемите (tokens) в програмния код са аналози на думите и препинателните знаци в текстовете на естествен език. Биват: разделители, литерали, служебни думи и идентификатори.

Скалата за измерване на характеристиките може да е:

1. **Номинална:** показва се наличието или отсъствие на някаква характеристика, без да се вземат предвид градациите.
- Пример (характеристика “**разбираемост**”).

Програмата може да бъде:

- *Разбираема*
- *Неразбитаема*

2. **Ранжирана (Порядкова)**: позволява ранжиране на някои характеристики чрез сравнение, с опорно значение.

- Пример (характеристика “**разбираемост**”).
Програмата може да бъде: **лесна, умерено трудна, трудна и много трудна** за разбиране.

По тази скала може да се определи и взаимното положение на програмите, намиращи се на едно ниво:

- Пример (характеристика “**разбираемост**”).
Програмите А , Б и В се отнасят не само към “умерено трудните”, но програмата “**Б е по-трудна от А , а А е по-трудна от В**” , т.е. $B > A > V$.

3. **Относителна**: позволява не само ранжиране на някои характеристики на обекта чрез сравнение, с опорно значение (**програмите А и В са трудни**), и не само да се определи взаимното положение на програмите на едно ниво (**но А е по-трудна от В**), но и да се определи точно на какво разстояние се намират обектите, които са на едно ниво (н.р - **трудни**):

- Пример (характеристика “**разбираемост**”).

Програмите „А и В са трудни за разбиране, но програмата А е 2 пъти по-трудна от В“.

4. Интервална: характеризира се с относителни или абсолютни величини в даден интервал.

Пример (характеристика “размер”):

Програма **А** е с размер между **2100KLOC**, а
В - между **2100-2200KLOC**.

Коментарите в програма **А** са **120LOC**, а в
програма **В** – **240LOC**

Или:

Програма **А** е с размер между **2000-2100KLOC**, а
В - между **2100-2200KLOC**.

Коментарите в програма **А** са **20%** от размера,
а в програма **В** - **40%**.

2. „Измерването“ в софтуерното производство

Измерването и получаваните мерки трябва да притежават следните свойства:

- **Обективност** — получаваните мерки да не зависят от субекта, извършващ измерването.
- **Надежност** — мярката да има необходимата различаваща способност и при повтаряне на измерванията при еднакви условия да се получават еднакви резултати.
- **Валидност** — получаваните мерки да отразяват реално свойствата на измервания обект.

2. За „Измерването“ в софтуерното производство

Кога е основателно да го правим?

- а) ако ще можем точно да измерим някакво свойство (характеристика) на обект в софтуерното производство (обект в софтуерното производство = спецификация, проект, програма, документация, процес, ресурс и др.);
- б) ако това свойство е съществено за обекта, и чрез неговото измерване може да направим съответните изводи за софтуерния обект (продукт, процес или ресурс);
- в) ако връзката между свойството и обекта е потвърдена (т. е. доказана е теоретично и експериментално) и може да бъде изразена чрез формула или в термините на съответен модел.

2. За „Измерването“ в софтуерното производство

Следователно основните проблеми при създаване на софтуерни метрики са два:

1. Да се:

- идентифицират съществени за изследване характеристики на софтуерни обекти и те
- да се дефинират в изчислими термини (т.е да са налице формулите за изчисляване);

2. Да се определят:

- типът на използваната скала,
- допустимите операции върху метриките,
- статистическият смисъл на оценяването и
- ограничаването на грешките при измерване.

Какво следва в тази лекция?

- Предложени са класификации на софтуерните метрики по различни критерии.
- Изброени са (на следващия слайд) и са описани няколко софтуерни метрики, които се смятат за „класически“. Те измерват качествени характеристики (сложност, стилистика и разбираемост на програмата) на софтуерни продукти, основно – програми.
- Накрая са разгледани някои методологични проблеми на изграждане на система от метрики в конкретна софтуерна организация.

Класически софтуерни метрики

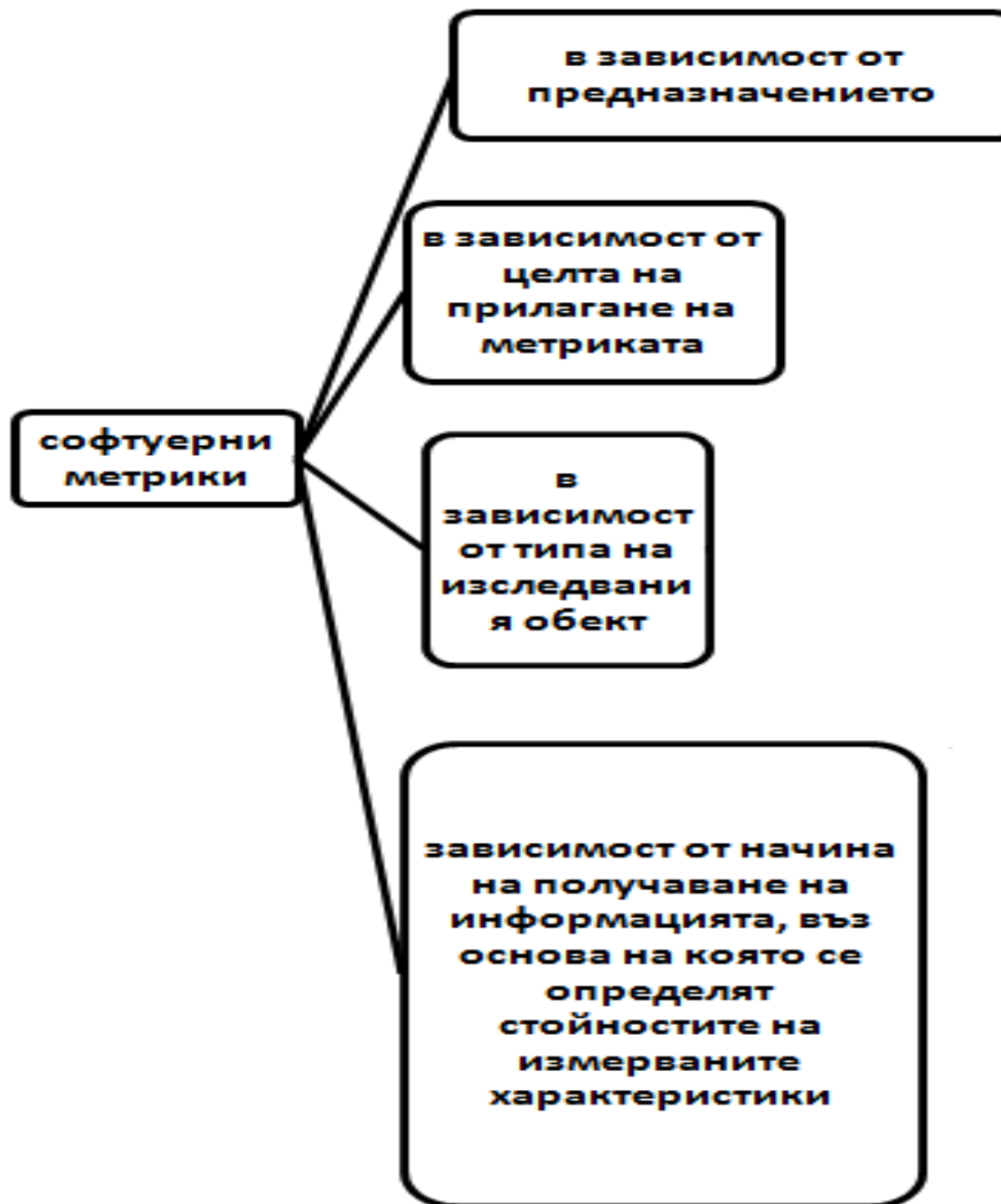


Класически софтуерни метрики

- метрика за размера на програмата,
- метрика на Маккейб за структурна (цикломатична) сложност и нейното разширение: метриката на Гонг—Шмид),
- метрика на Холстед за текстуална сложност,
- метрика за функционална плътност на програмата
- метрика на Рехенберг за технологична сложност,
- метрика на Джилб за логическата сложност на програмата,
- метрика на граничните стойности - за оценка на качество на програмата,
- метрика, базирана на обръщенията към глобалните променливи в програмата,
- метрика на Чепин – за определяне характера на използваните променливи в програмата,
- метрика за стилистика и разбираемост на програмата,
- метрика за четимост на други видове софтуерни продукти: ръководства за потребителя, материали за обучение, спецификации и др.)

3. Класификация на софтуерните метрики

- Известни са около **250 софтуерни метрики**. Изключително голямото разнообразие на съществуващите софтуерни метрики затруднява реалното им използване.
- Следват класификации на метриките най-общо, по няколко критерия, и за всеки критерий поотделно.
- Основно внимание се отделя на метриките, свързани с продукта “програма”.



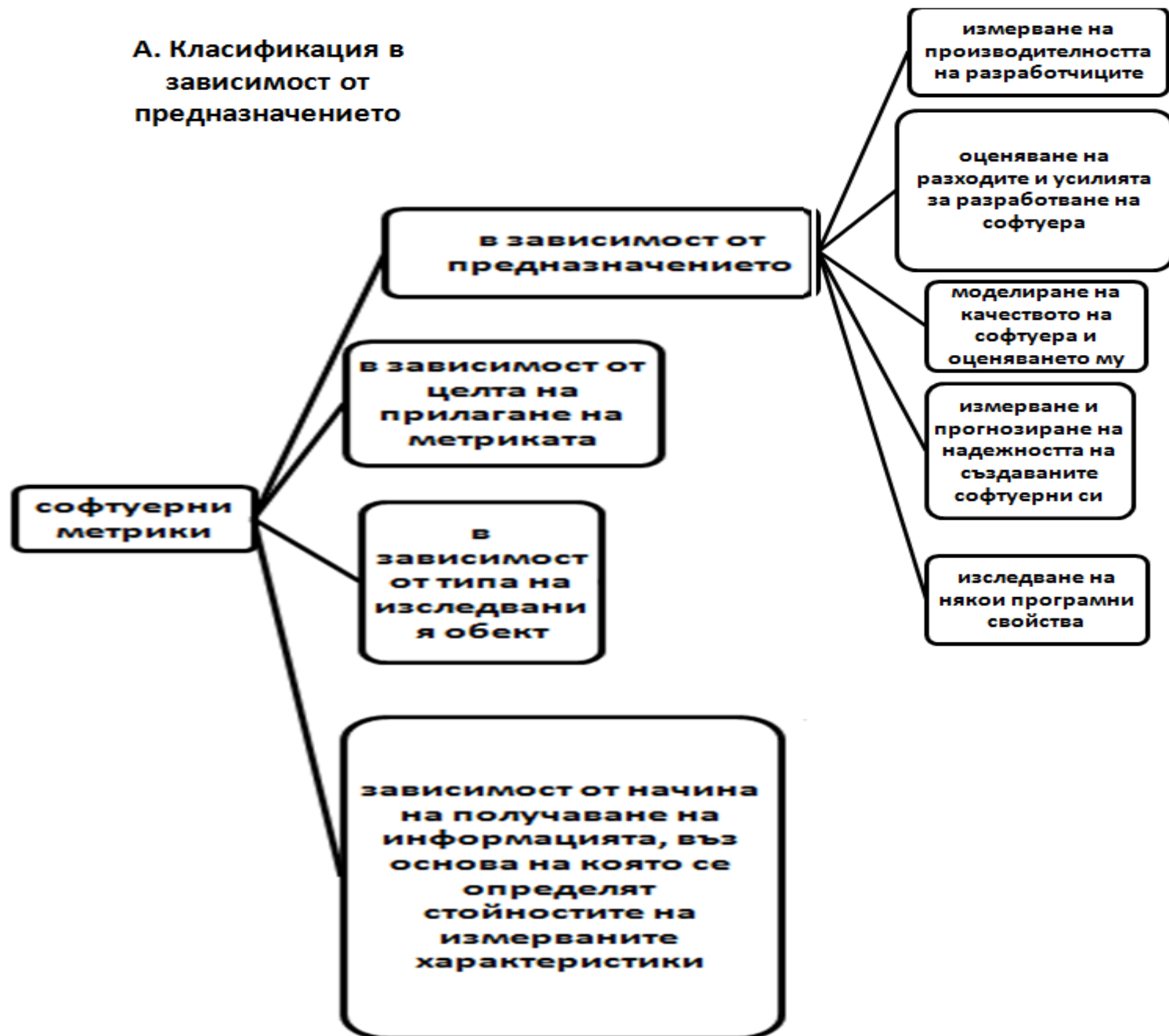
Обща класификация на софтуерните метрики, по критерии

А. Класификация в зависимост от предназначението

Най-често софтуерните метрики се използват за:

- а) оценяване на **разходите и усилията за разработване на софтуера**;
- б) измерване на **производителността на разработчиците**;
- в) моделиране на **качеството на софтуера** и **оценяването му**;
- г) измерване и прогнозиране на **надежността на създаваните софтуерни системи**;
- д) изследване на **някои програмни свойства**.

**А. Класификация в
зависимост от
предназначението**

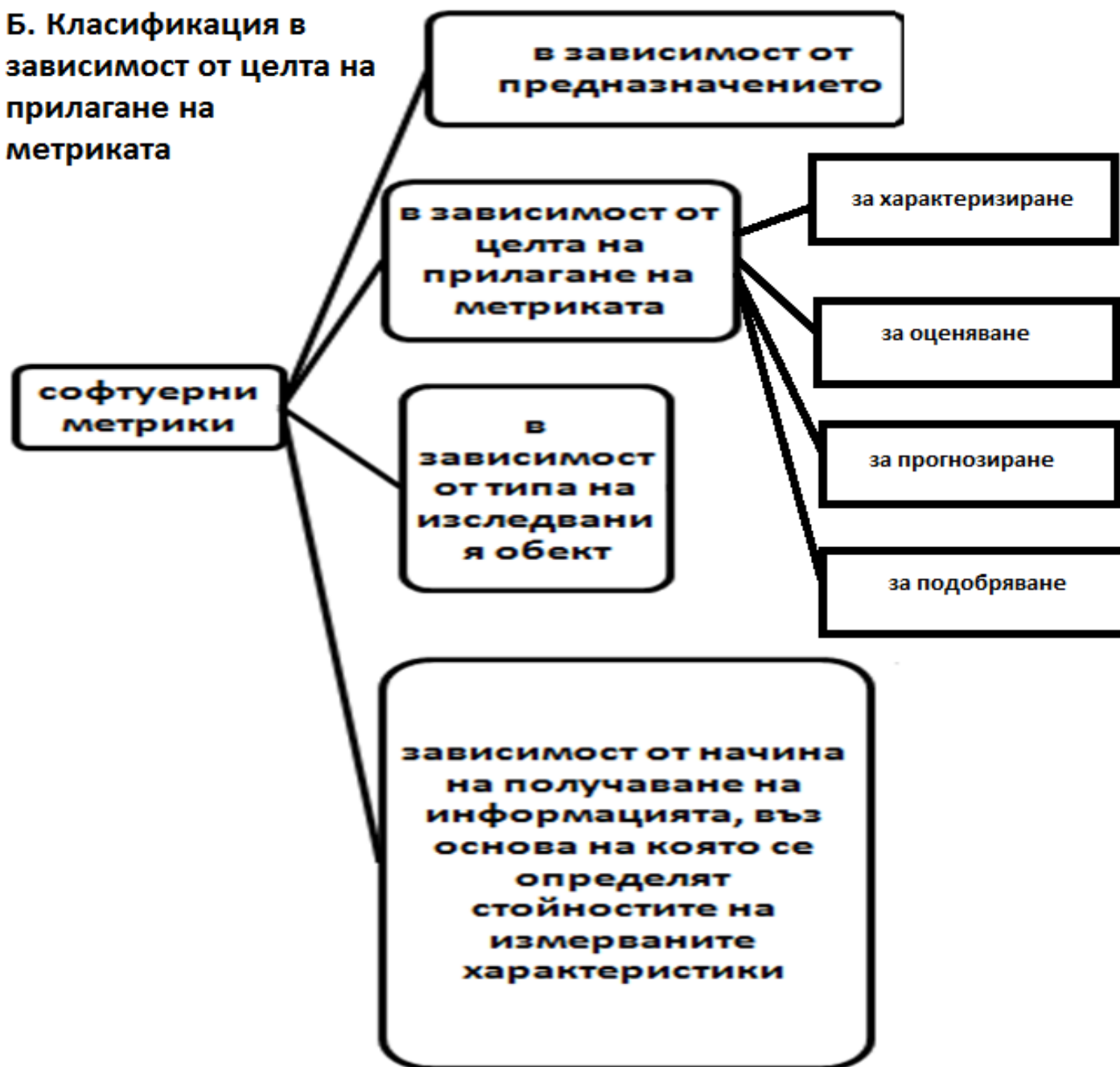


Б. Класификация в зависимост от целта на прилагане на метриката

Софтуерните метрики могат да бъдат:

- а) за характеризиране (to characterize) — **да разберем изследвания обект** и да установим база за сравнение при бъдещи оценки;
- б) за оценяване (to assess) — **регистриране на текущо състояние** на изследвания обект;
- в) за прогнозиране (to predict) — **предварителна оценка за бъдещо състояние на изследвания обект**;
- г) за подобряване (to improve) — **натрупване на количествена информация** за определяне на факторите, от които зависи качеството на продукта или ефективността на процеса.

Б. Класификация в
зависимост от целта на
прилагане на
метриката



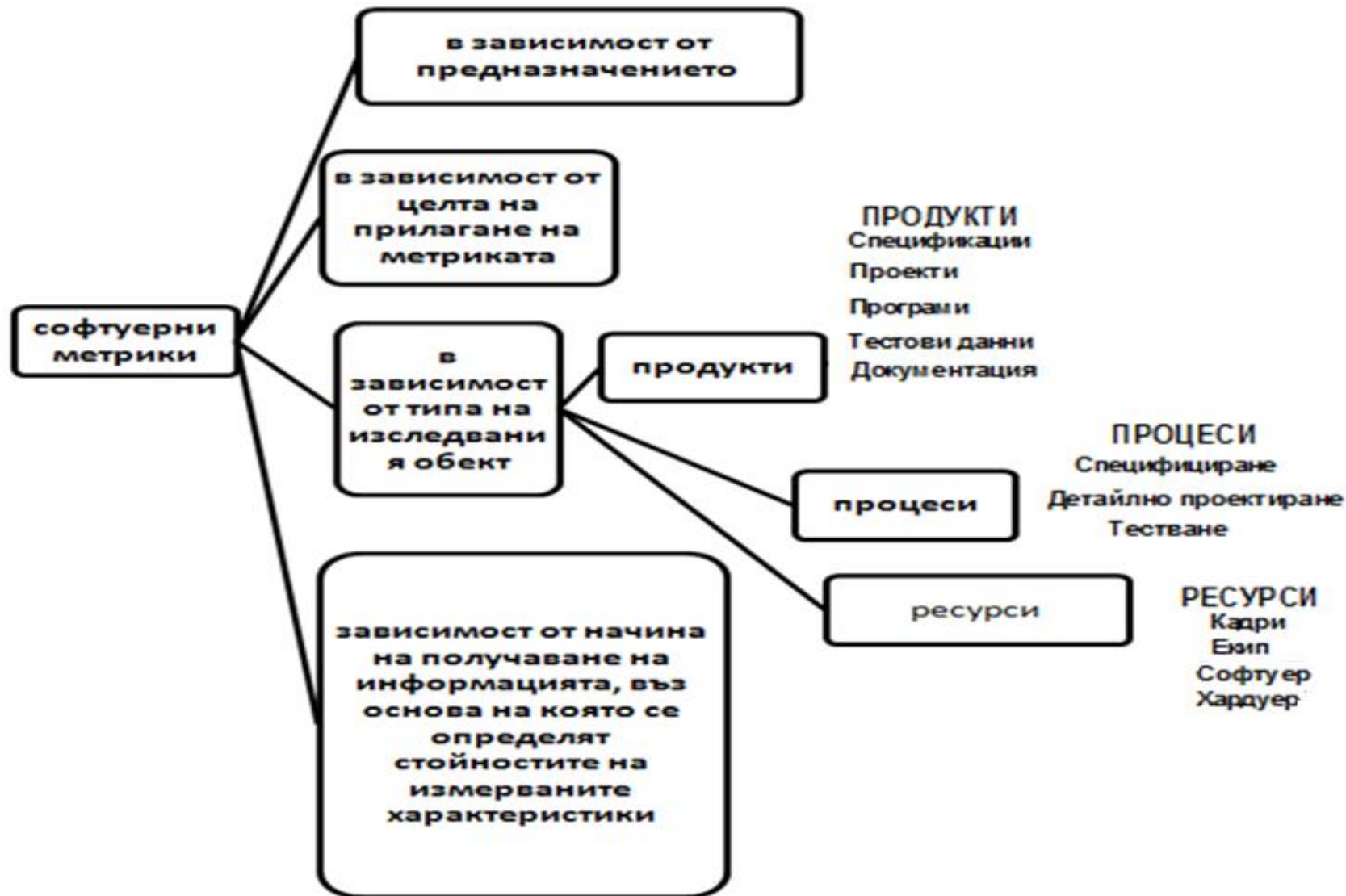
В. Класификация в зависимост от типа на изследвания обект

В софтуерното производство могат да се разграничат три основни типа **обекти**:

- а) **процеси** — дейности, свързани със създаването и използването на софтуера например специфициране, проектиране, тестване и др.;
- б) **продукти** — резултати от процесите, например спецификации, проекти, програми, документация и др.;
- в) **ресурси** — елементи, входни данни за процесите, например кадри, хардуер и др..

Всичко, което можем да измерваме, е характеристика на обект от горните три класа.

В. Класификация в зависимост от типа на изследвания обект



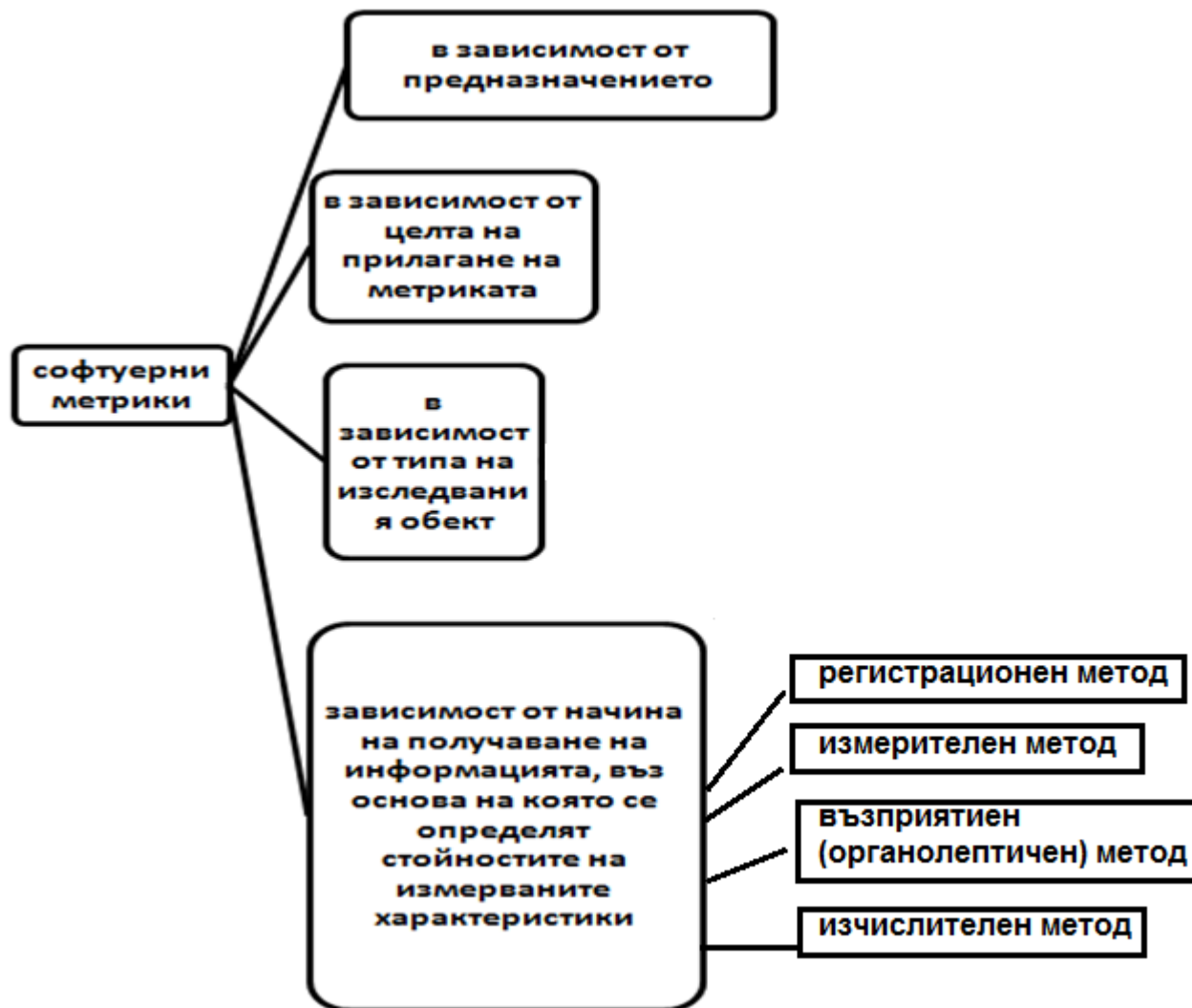
- Измерваните характеристики могат да бъдат:
 - **вътрешни** (значими за самия обект, например ако обектът е „документация“ такива характеристики са „обем“ и „пълнота“) или
 - **външни** (значими в средата, в която обекта се използва, например ако обектът е „документация“ такива характеристики са „разбираемост“ и „адекватност“).
- Примери за обекти в софтуерното производство и съответните им вътрешни и външни характеристики са описани в таблицата на следващия слайд (структурата на таблицата и данните са взети от [5]).

ОБЕКТИ	ВЪТРЕШНИ ХАРАКТЕРИСТИКИ	ВЪНШНИ ХАРАКТЕРИСТИКИ
ПРОДУКТИ Спецификации Проекти Програми Тестови данни Документация ПРОЦЕСИ	Размер Модулност Коректност Функционалност Възможност за повторно използване Размер Модулност Свързаност Размер Алг. сложност Тестируемост Размер Ниво на тестване Обем Пълнота	Изчерпателност Съпровождаемост Разбираемост Качество Сложност Съпровождаемост Надежност Приложимост Съпровождаемост Качество Изчерпателност Разбираемост Адекватност
Специфициране	Продължителност Трудоемкост Брой изменения в изискванията	Качество Разходи Стабилност
Детайлно проектиране	Продължителност Трудоемкост Брой открити грешки в спецификациите	Разходи
Тестване	Продължителност Трудоемкост Брой открити грешки в спецификациите	Разходи Изчерпателност Систематичност
РЕСУРСИ		
Кадри	Възраст Възнаграждение	Производителност Професионален опит
Екип	Численост Структура	Продуктивност Качество
Софтуер	Цена Размер	Ползност Надежност
Хардуер	Цена Изч. възможности	Надежност на функциониране

Г. Класификация на метриките, в зависимост от метода за получаване на информацията, нужна за определяне на стойностите на измерваните характеристики:

- а) **регистрационен метод** — получаване на информацията **по време на разработване или функциониране на ПП, чрез регистриране на отделни събития** — продължителност на сеанс, брой аварийни ситуации и описанието им в документацията и др.;
- б) **измерителен метод** — информацията се получава чрез прилагане на **специално разработени инструментални средства**;
- в) **възприятиен (органолептичен) метод** — информация, получена след **анализ на зрителни или слухови възприятия** — използвани цветове, бързина на смяна на текстове, звукова сигнализация и др.
- г) **изчислителен метод** — използване на **теоретични или емпирични зависимости**, изведени въз основа на статистическите данни, натрупани през различни фази на жизнения цикъл на ПП.

Г. Класификация в зависимост от начина на получаване на информацията, въз основа на която се определят стойностите на измерваните характеристики:



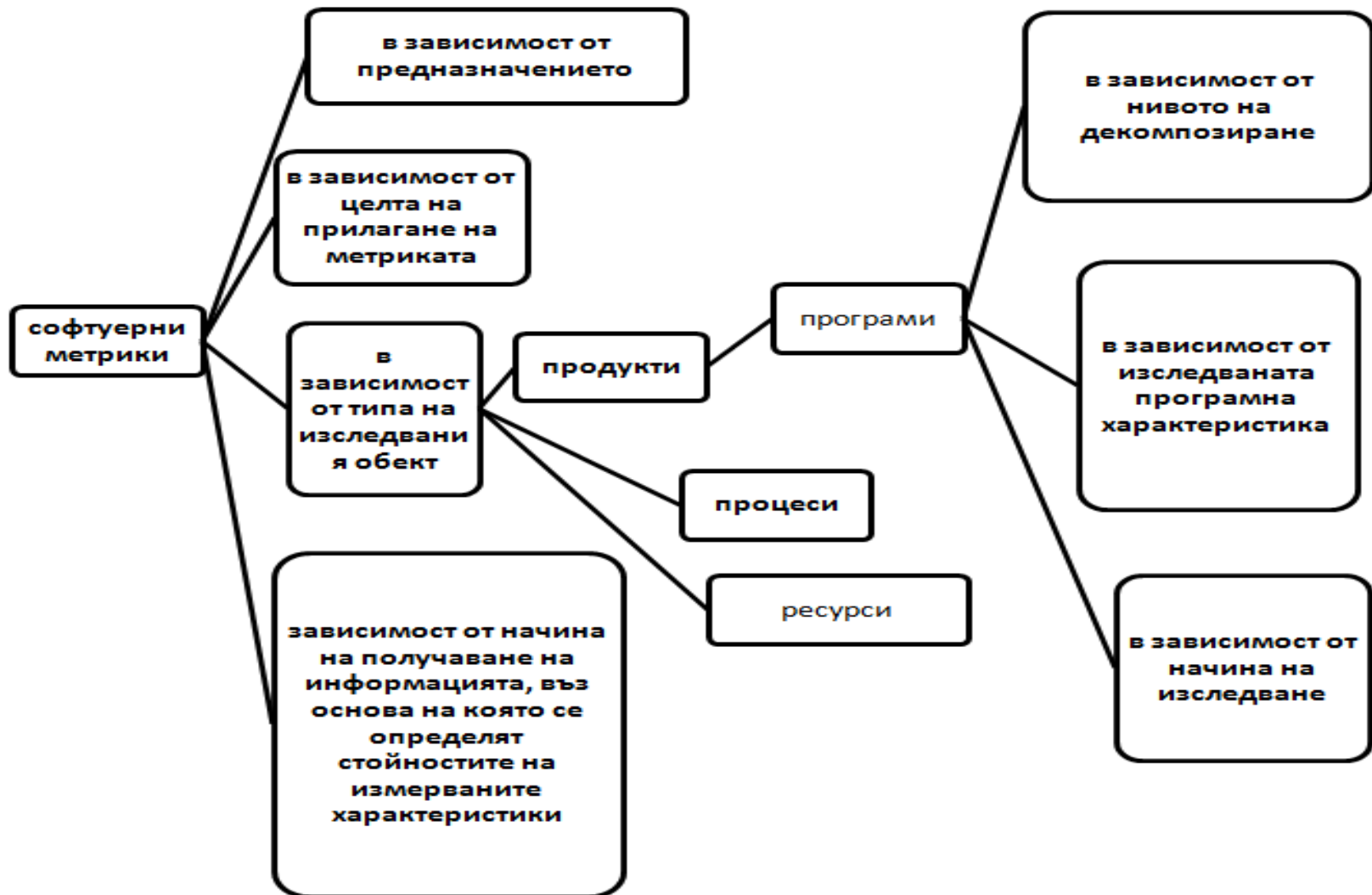
Д. Класификация на метриките, свързани с продукт „програма”

- Метриките, свързани с изследване на характеристики на „програми” са се появили първи. Причините:
 - Програмите **са най-важният продукт**, създаван във всеки софтуерен проект.
 - **Сравнително лесно**, дори за неспециалист в областта на софтуерните метрики, е **идентифицирането на програмни свойства**, които да бъдат изследвани с определена цел.
- За този вид метрики са възможни следните класификации:

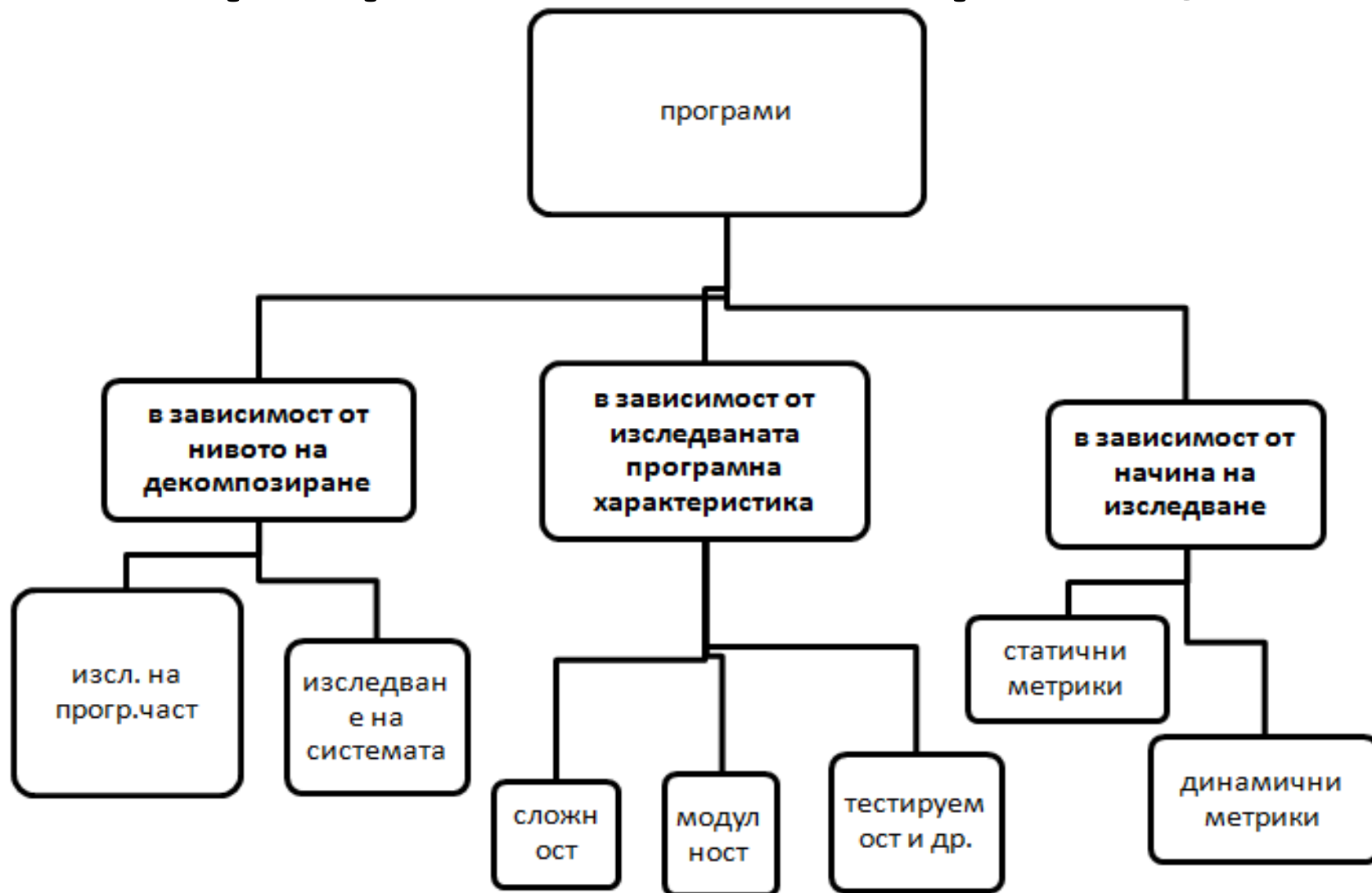
Д. Метрики, свързани с продукт „програма” - класификации

- а) **в зависимост от начина на изследване** — имаме статични и динамични метрики.
 - При статичните метрики се анализира само текстът на програмата, докато за динамичните метрики е необходимо изпълнението ѝ.
- б) **в зависимост от изследваната програмна характеристика** — сложност (структурна, текстуална или алгоритмична), модулност, тестируемост, независимост от средата, модифицируемост и др.
- в) **в зависимост от нивото на декомпозиране** — изследване на отделна програмна част или на програмна система като цяло. Метриците от втората група са много по-сложни, защото се изследва потокът на данните и потокът на управление в многокомпонентни системи.
 - Една метрика се нарича **адитивна**, ако резултатите от прилагането и за програмна система могат да се получат чрез сумиране на резултатите от прилагането и върху съставлящите я програмни части.

Д. Метрики, свързани с продукт „програма” - класификации



Д. Метрики, свързани с продукт „програма” - класификации



4. Примери за софтуерни метрики

М1. Метрика за размера на програма (LOC)

I. Описание

- Метриката е **статична** и може да се **прилага както за отделен програмен модул, така и за програмна система.**
- За мярка на размера на програма се избира броят програмни редове **LOC** (Lines Of the Code) в изходния текст на програмата.

M1. Метрика за размера на програма (LOC)

$LOC = L1 + L2$ където

LOC — общ брой редове;

L1 — коментарни и празни редове;

L2 — същински програмни редове.

- Метриката е адитивна.

II. Приложимост на метриката

- LOC е най-проста мярка за извършената от програмиста работа.
- Броят на грешките, усилията за разбиране (т.е разбираемост), поддържане и съпровождане (т.е съпровождаемост) са правопропорционални на размера на програмите.
- В зависимост от размера (на ниво програмна единица) и на размера и броя модули в програмна система програмите могат да се класифицират като:
 - **прости,**
 - **средно сложни,**
 - **сложни и**
 - **свръх-сложни.**
- За всяка категория могат:
 - да се формулират мерките за осигуряване на **качеството,**
 - да се оценят обхватът и **сложността на тестването,**
съпровождането и др.

Проблеми: Няма единство в разбирането за това, какво е **ред** на програмата: функционалния оператор, операторите за декларация на променливите или всички оператори, включително и коментарите. Най-често се взема функционалният оператор, тъй като в него се влага творчество и е носител на интелект.

- Често пъти, вместо LOC се прилага производната мярка – **KLOC** (Kilo Lines Of the Code) – хиляди редове от програмата.

Предимства

- Лесно се прилага и лесно се разбира
- Приемлива за класификация на програми, съществено различаващи се по обем.

Недостатък

- Оценката за размера на програмата е недостатъчна за оценка на сложността на програмата.

М2. Метрика на Маккейб за структурна (цикломатична) сложност на програма

I. Описание

- Изключително цитирана в литературата статична метрика, измерва сложността на потока на управление в програмата.
- Базира се на **управляващия граф $G = (V, E)$** на програмата, чийто “ n ” ($n = |V|$) на брой върхове представят управляващите оператори на програмата, а дъгите (имат посока, “ e ” на брой, $e = |E|$) – предаването на управление между съответстващите на върховете, оператори на програмата.

Тогава $V(G) = e - n + 2p$, където:

- $V(G)$ е цикломатичната сложност, представляваща максималния брой линейно-независими пътища в програмата;
 - e — брой дъги в управляващия граф;
 - n — брой върхове в управляващия граф;
 - p — брой на компонентите на свързаност в графа.
- Числото p може да се разглежда като броя на дъгите, който трябва да се добави към управляващия граф, за да се преобразува той в силно свързан граф.
- **Силно свързан е този граф, при който, всеки два върха са достижими.**
- За графите на коректните програми силно свързан граф се получава, по правило, чрез свързване с дъга **на върха за край** на програмата **с върха за началото**.

- По същество $V(G)$ определя броя на «**линейно - независимите контури**» в силно свързания граф, което има значение при разработване на тестове за продукта (**счита се, че $V(G)$ е най-добре да бъде до 10 и да не надвишава 15**).
- Цикломатичното число на Маккейб показва необходимите проходи (пътища) за покриване на всичките контури на силно свързания граф или количеството на тестовите "прогонвания" на програмата, необходими за изчерпващо тестване на критерия: **"работи всеки клон"**.
- Разработени са инструментални средства за автоматично изчисляване на цикломатичната сложност, като при това се установяват и някои структурни неправилности — недостижими програмни части, пресичащи се цикли и други.

I. Недостатъци на Метрика на Маккейб

- Недостатъци на метриката са, че не отчита
 - дължината на линейните участъци и
 - нивото на влагане на управляващите структури.
- Не се отразяват и междумодулните връзки.
- Разширение на тази метрика е **метриката на Гонг—Шмид** [6], която предлага цикломатичната сложност да се изчислява по формулата
$$C(G) = V(G) + E,$$
- E - представя обобщената степен на вграждане на управляващите структури една в друга.

II. Приложимост на Метрика на Маккейб

Като мярка за сложността на модула

- Метриката предлага мярка за **модулна сложност**, която служи за оценка на такива характеристики, като разбираемост на програмите и съпровождаемост.

За класификация на модулите по сложност:

- **$V(G) < 10$ – прости**
- **$10 \leq V(G) < 20 \div 30$ – средно сложни**
- **$20 \div 30 \leq V(G) < 50$ – сложни**
- **$V(G) > 50$ – недопустимо сложни.** Трябва да се обмисли реструктуриране (разделяне на няколко модула).

Прекъснатата дъга се прибавя
за образуване на силно
свързан граф.

При този граф:

$e = 8; v = 6; p = 1;$

$$V(G) = 8 - 6 + 2 = 4;$$

Имаме силно свързан граф с
четири линейно-независими
контури:

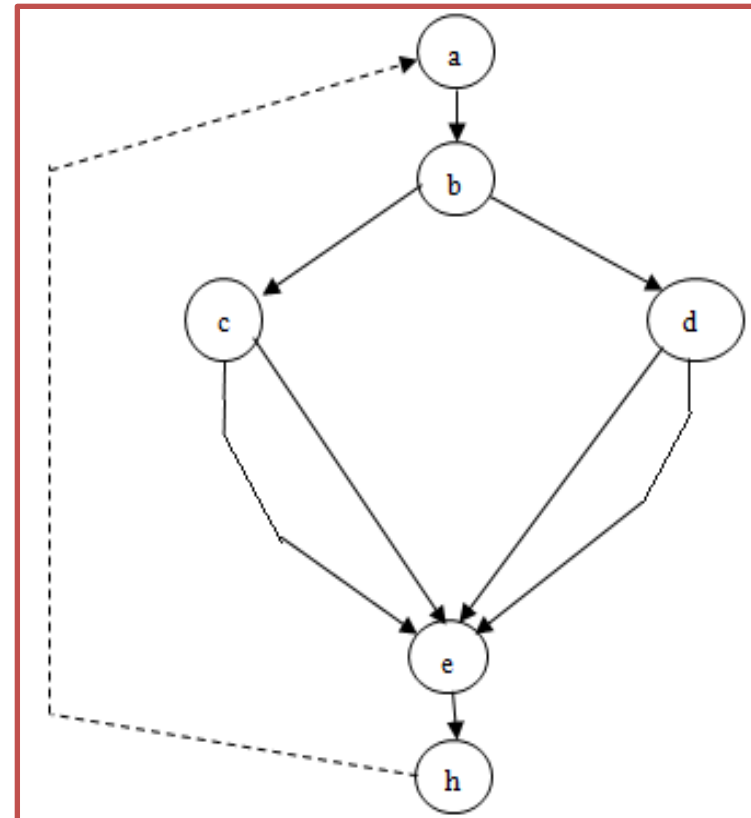
$a - b - c - e - h - a;$

$a - b - c - e - h - a;$

$a - b - d - e - h - a;$

$a - b - d - e - h - a;$

**М2. Метрика на
Маккейб за
структурна
(цикломатична)
сложност - пример**



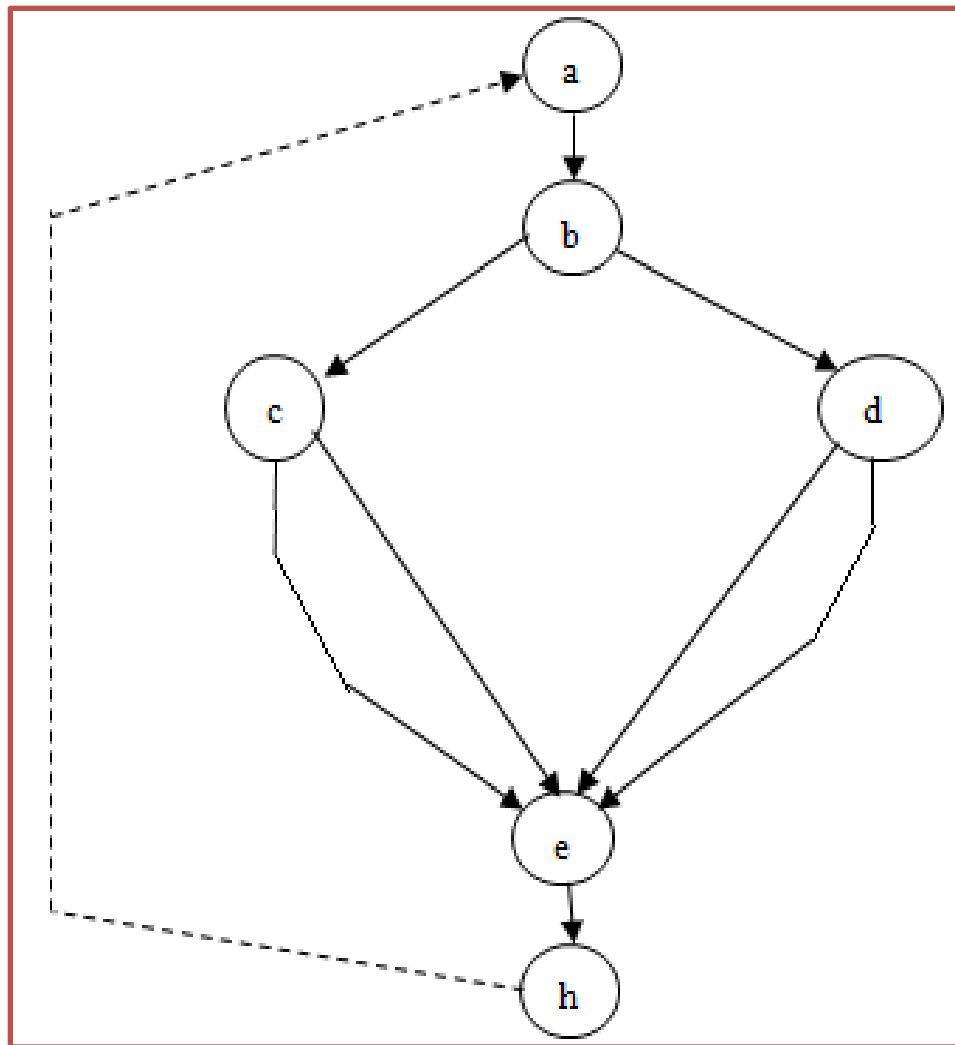
- По определение за управляващия граф на програмата върховете **(a, b)** и **(e, h)** би следвало да се обединят, но това по същина не изменя нещата, тъй като след сливането :

$e = 6; v = 4; p = 1$ и

$$\mathbf{V(G) = 6 - 4 + 2}$$

т.е. **$V(G)$** остава без изменение.

- Цикломатичното число зависи само от количеството на предикатите, като сложността им не се отчита.



- Например ако имаме два условни оператора:

IF (X > 0) THEN X = A ELSE X = B;

IF ((X > 0) AND (FLAG = '1')) OR ((X = 0 AND (FLAG = '0')) THEN X = A ELSE X = B;

- то и двата оператора ще се представят с един и същ граф:

- Ясно е, че **цикломатическото число за двата оператора ще бъде едно и също** (графът им е еднакъв) и **няма да отразява сложността на предиката**, което е твърде съществено при оценката на програмата.



МЗ. Метрика на Холстед за **текстуална** **сложност на програма**

I. Описание

- Статична метрика за програма, написана на произволен език за програмиране. Предложена е от Холстед през 1976 г. [7] и е метрика, за която има най-много публикации.
- Програмата се разглежда като състояща се **от активни елементи (оператори) и пасивни елементи (операнди)**. Операндите са константи или променливи, използвани при реализацията на алгоритъма в конкретна програма. Операторите са елементи, влияещи върху стойностите или наредбата на операндите.

За прилагане на метриката се определят:

- n_1 - брой на уникалните оператори. Броят на уникалните оператори определя речник на операторите.
- n_2 - брой на уникалните операнди. Те образуват речник на уникалните операнди;
- N_1 - общото количество на операторите в програмата;
- N_2 - общо количество на операнди в програмата.

Използвайки тези характеристики се получават следните оценки :

- **речник** на програмата: $n = n_1 + n_2$;
- **дължина** на програмата : $N = N_1 + N_2$;

- Изчислена дължина на програмата:

$$N = n_1 \cdot \log_2 n_1 + n_2 \cdot \log_2 n_2$$

- Изчислен обем на програмата:

$$V = (N_1 + N_2) \cdot \log_2 (n_1 + n_2) = N \cdot \log_2 (n)$$

- Трудност на програмата: $D = (n_1 / 2) \cdot (N_2 / n_2)$

- Ниво на програмата: $L = 1 / D$

- Усилия за създаване на програмата: $E = V \cdot D$

- Преброяването на n_1 , n_2 , N_1 и N_2 , както и изчисляването на метриките за гореописаните свойства, може да се извършва от инструментално средство (специализиран лексически анализатор).

II. Приложимост на метрика на Холстед за текстуална сложност

- Четирите оценъчни елемента и сравнително простата и лесна за автоматизиране процедура за пресмятане, правят тази метрика с изключително високо ниво на практическо използване, н.р за:

- 1. За оценка на времето за програмиране T по формулата $T = E/S$, където E е усилието, в “човеко-месеци” - ЧМ) за разработка на програмата, а S е индекс на Страуд за интензивността на интелектуалните занимания, който за програмирането има стойност $S = 18$. Така:**

$$T = E/S = E/18$$

2. За оценка на усилията за разработването на програмата. За оптимална се счита сложност $E_{op} = 60\ 800[ЧМ]$, за максимална - $E_{max} = 129\ 600[ЧМ]$.

- Сравняването на E за конкретна програма с тези две стойности дава възможност да се класифицират усилията за разработването и като малки, средни или големи.

3. За оценка на усилията за съпровождане. Гордън доказва [9], че усилията за създаване на нова програма са сравними с усилията за разбиране на съществуваща. Този факт често се използва за **предварителна оценка на усилията за съпровождане** в случаите, когато то не се осъществява от разработчиците.

М4. Метрика за функционална плътност на програмата

Към метриците за текстуална сложност може да се отнесе и функционалната плътност на програмата (FD), определена по следния начин.

- $FD = LOC/FP$,
- където **LOC** е общият брой редове в програмата (Lines Of Code);
- **FP** (Function Points) – брой функционални точки (брой на извиквания на функции).

Анализите сочат следните стойности:

- стандартна стойност – 62;
- идеална стойност - 36;
- диапазон – < 25 до 150.

М5. Метрика на Рехенберг за технологична сложност

I. Описание

- Метриката на Рехенберг [11] е опит за преодоляване на едностранчивостта на съществуващите метрики за сложност. Тя е статична, комплексна метрика, която може да се прилага за програми или програмни системи, написани на произволен език за програмиране от високо ниво.

Рехенберг предлага следната формула:

- $CC = SC + EC + DC,$

където:

- CC е комплексната сложност;
- SC — операторна сложност;
- EC — сложност на използваните изрази;
- DC — сложност на данните.

Въведени са и относителни мерки за сложност.
Ако NS е броят на операторите в програмата, то
съответните относителни мерки са:

- $RCC = CC/NS$
- $RSC = SC/NS$
- $REC = EC / NS$
- $RDC = DC / NS$

Относителните мерки дават възможност да се
сравняват програми с различна дължина.

- **Операторната сложност SC** на програмата е сума от операторните сложности на съставлящите я оператори.
- За всеки тип оператор е въведена мярка за сложността му.
- Например:
 - операторът за присвояване има сложност 1,
 - операторите за цикъл — 3,
 - операторът `goto` — 5 и т. н.
- Влагането на операторите се отчита чрез коригиращия коефициент $k = 1.5$, с който се умножава мярката за сложност на всеки вложен оператор.

- **Сложността на изразите ЕС** се пресмята чрез мерки за сложността на използваните в израза операции.
 - Например операциите „+“ и „-“ и операциите за сравнение „>“, „=“, „<“ имат мярка за сложност 1.
 - Операциите „*“ и „/“ имат мярка за сложност 2, логическите операции и/или имат стойност 3 и т. н.
- **Сложността на данни DC** измерва разстоянието между декларирането и използването на данните. Мярката за сложност DC за програмата е сума от мерките за сложност на използваните променливи (локални и глобални).

II. Приложимост на Метриката на Рехенберг за технологична сложност

- Мярката на Рехенберг може да се използва за прогнозиране на усилията за тестване и съпровождане чрез сравняване на мерките на новоразработвана софтуерна система със съществуваща, за която тези усилия са документирани.

М6. Метрика на Джилб за логическа сложност на ПО

I. Описание

Логическата сложност на програмата се определя като наситеност на програмата с изрази от типа:

- IF - THEN - ELSE (или кратката форма IF-THEN).

В тази метрика се въвеждат две характеристики:

- **GL** - абсолютна сложност на програмата, характеризираща се с количеството условни оператори;
- **gl** - относителна сложност на програмата, характеризираща се с наситеността на програмата с условни оператори.

Относителната сложност на програмата се определя по формулата:

$$gl = \frac{GL}{Op}$$

- където **Op** - общият брой на операторите в програмата.

II. Приложимост

Тази метрика е **проста и ефективна** оценка на сложността на програмното осигуряване.

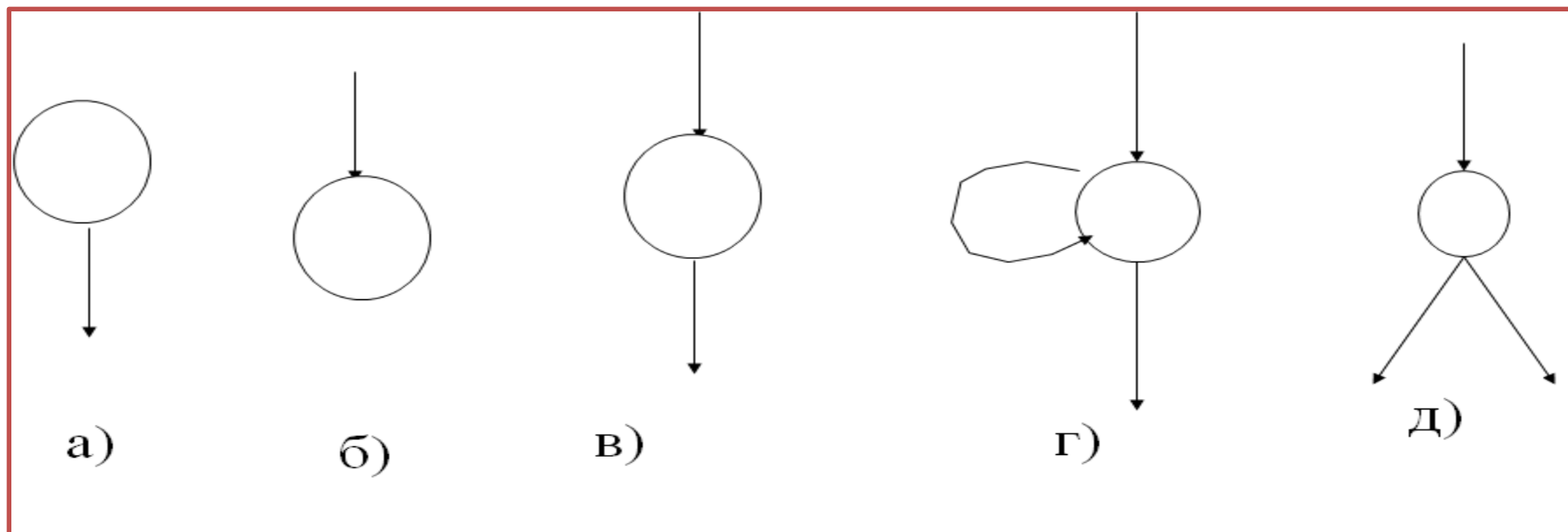
М7. Метрика на граничните стойности

I. Описание

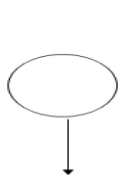
- Създадена от Изосимов [10]. Базира се на представяне на програмата като ориентиран граф и на следните понятия, свързани с последния.
- Нека $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ да е ориентиран граф, представящ програмата, с единствени начален и краен връх.

- Съвкупността (множеството) от върховете на графа $G = (V, E)$ се разбива на няколко типа.
- На всеки връх от тези групи може да се присвои определен коефициент, съответстващ на неговата сложност.

Имаме:



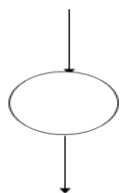
- **начален връх:** има само един изход - коефициент на **сложност 0** (а);
- **краен връх:** има само един вход – коефициент на **сложност 0** (б);
- **приемащ връх** - има един или повече входа и само един изход – коефициент на **сложност 1** (в);
- **връх с обратна връзка (цикъл)** – коефициент на **сложност 2**. (г);
- **алтернативен връх** - това е връх, чийто изходи са поне с един повече от входовете му (д). Върховете от този тип са начала на подграфи.



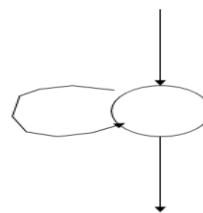
а)



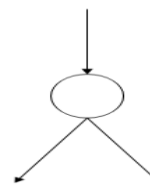
б)



в)



г)



д)

Каква сложност да се придаде на един алтернативен връх?

- Отговорът на този въпрос не е еднозначен:
 - Това зависи от сложността на подграфа, чието начало е даденият алтернативен връх.
 - Подграфът може да съдържа различно количество върхове с различна сложност.
- Ето защо за алтернативните върхове се въвежда така наречената **коригирана** или **приведена** сложност.

- За един алтернативен връх **k** коригираната сложност се определя чрез сумата:

$$S_k = \sum_{i=1}^n k_i ,$$

- където **n** – брой върхове от съответния подграф, определен от алтернативния връх **k**;
- **k_i** – сложност на **i** – тия връх в подграфа (чието начало е дадения алтернативен връх **k**):

$$k_i = \begin{cases} 0 - \text{краен връх и всеки връх, който не е приемащ;} \\ 1 - \text{приемащ;} \\ 2 - \text{връх с обратна връзка.} \end{cases}$$

- Сложностите на всички върхове от граф **G** се сумират, образувайки абсолютната гранична сложност на програмата.

$$S_a = \sum_{j=1}^m l_j ,$$

- където **m** – брой на върховете на изходния (първоначалния) граф, включително и алтернативните върхове;
- l_j – сложност на j -ия връх, отчитайки приведените стойности на алтернативните върхове.

относителната гранична сложност

- За сравнение на различни програми, оценени по дадената метрика, се въвежда

относителната гранична сложност:

$$S_o = 1 - \frac{v - 1}{S_a},$$

- Където S_o – относителна гранична сложност на програмата;
- S_a - абсолютна гранична сложност;
- v – общо количество на върховете на графа на програмата .

II. Приложение на метриката на граничните стойности

- За получаване на **оценка за сложността на програмата** по метода на граничните стойности е необходимо графът **G** да се разбие на максимален брой подграфи **G'**, удовлетворяващи следните условия:
 - вход в подграфа се осъществява само чрез алтернативни върхове ;
 - всеки подграф включва връх (**долна граница** на подграфа), в който може да се попадне от всеки друг връх на подграфа.
- Така алтернативен връх, свързан сам към себе си с дъга-обратна връзка, образува сам подграф.

Метрика на граничните стойности - пример

- В примерния граф, представящ програма, върховете **a, b, c** и **d** са алтернативни, **l** е краен, а всички останали са приемащи. След като имаме 4 алтернативни върха, то могат да се посочат и 4 подграфа, а именно:

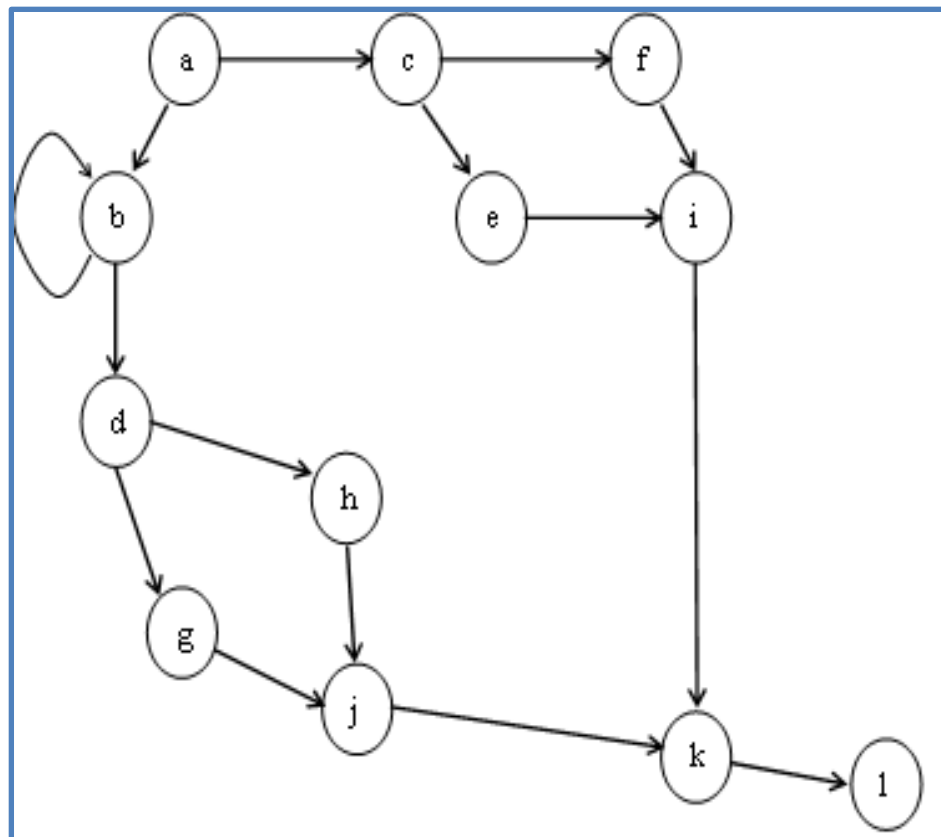
1. **a, b, c, d, e, f, g, h, i, j, k,**

Където **a** - алтернативен връх;
k – долна граница на подграфа.

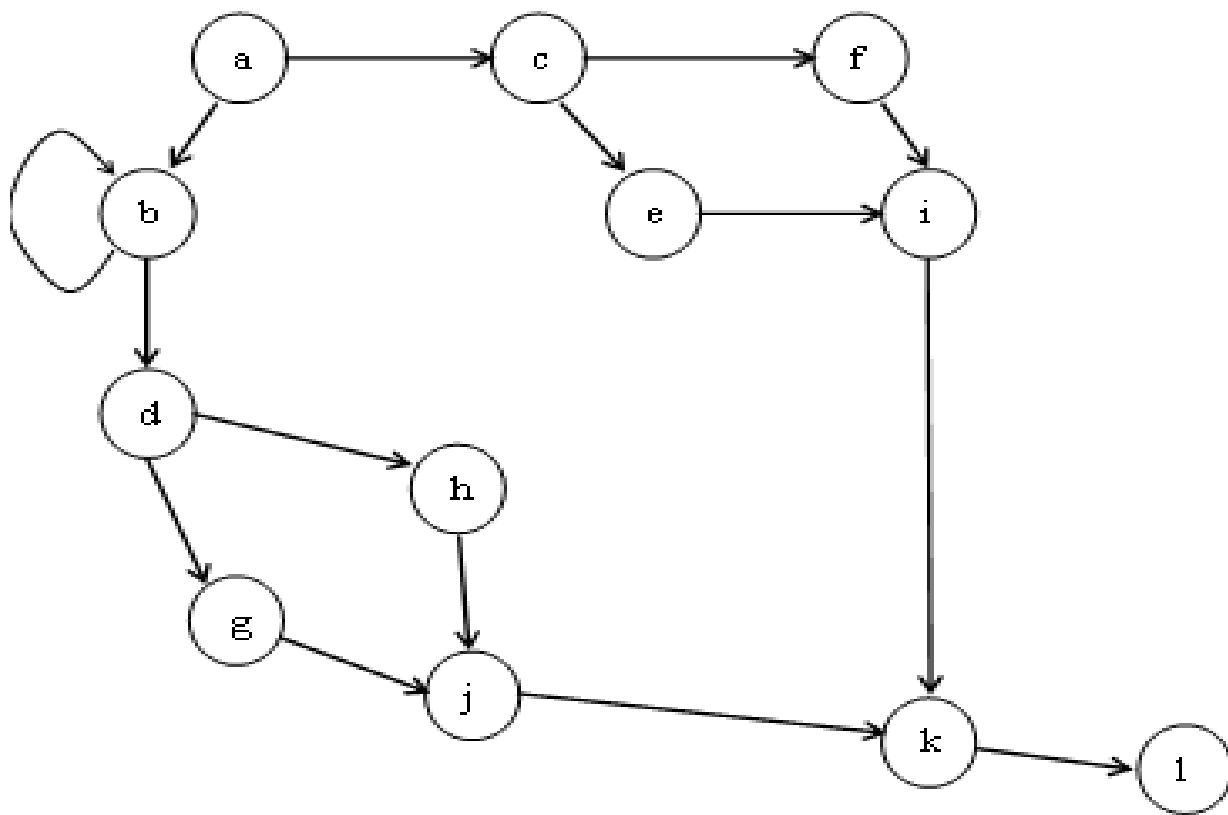
2. **b.**

3. **c, e, f, i.**

4. **d, g, h, j.**



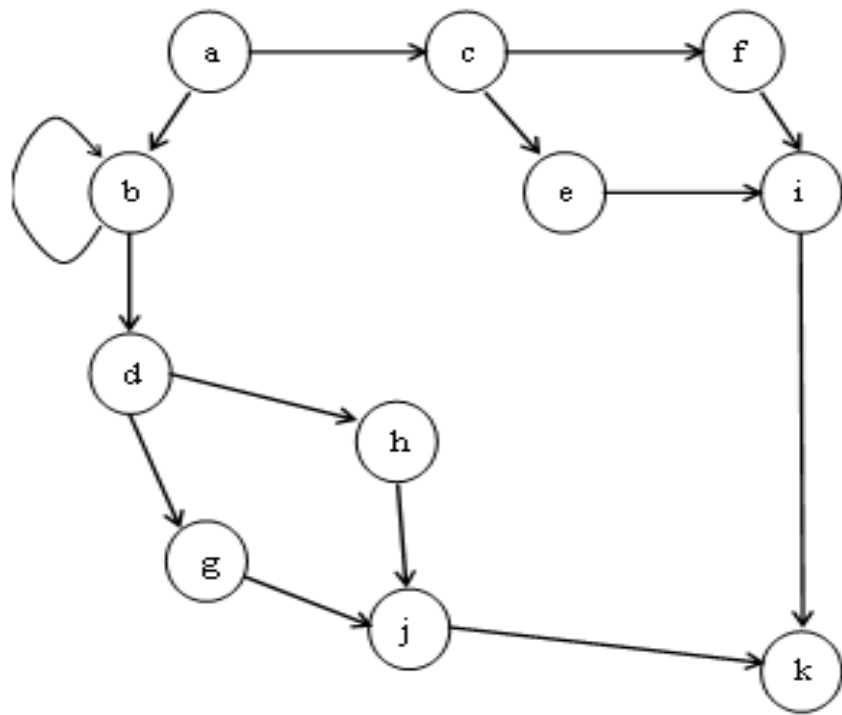
- Коригираната сложност на върха на всеки подграф се определя като сума от коригираните стойности на върховете, които участват в подграфа, като:
 - приемащият връх има коригирана стойност, равна на 1;
 - крайният връх има коригирана стойност, равна на 0;
 - всеки връх с обратна връзка има коригирана стойност, равна на 2;



Коригираната сложност на

подграф 1 ще бъде 10, понеже:

- връх **a** е начален, следователно неговата коригираната сложност ще бъде равна на 0;
- връх **к** е краен, следователно коригираната му сложност ще е равна на 0;
- върхът **b** е връх с обратна връзка следователно ще има коригирана сложност, равна на 2;
- върховете **c, d, e, f, g, h, i, j** са приемащи върхове, т.е. тяхната коригираща сложност ще бъде равна на 1.

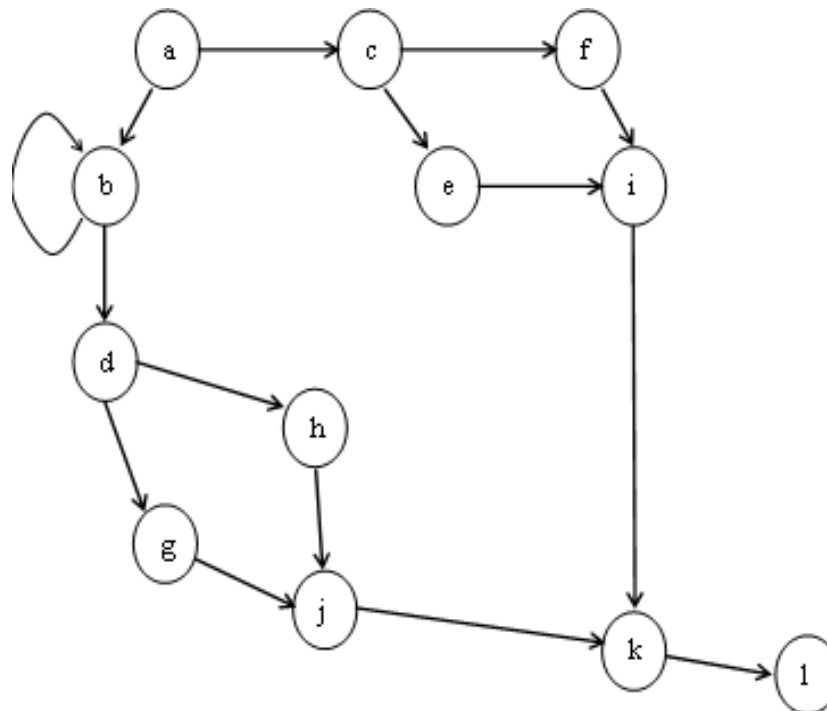


Подграф 1:

**a , b , c , d , e , f , g , h ,
i , j , k,**

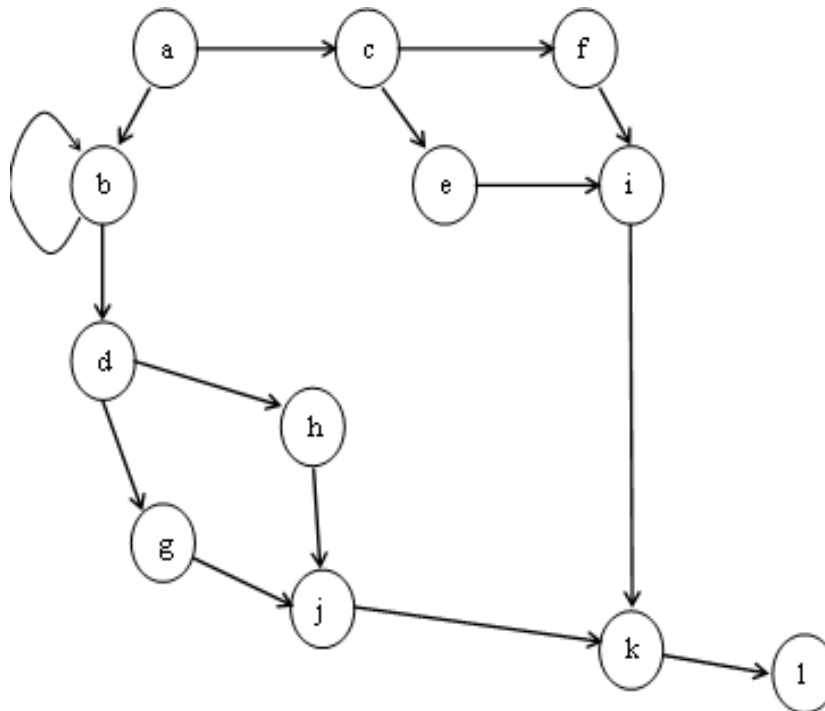
Метрика на граничните стойности - пример

- Коригираната сложност на подграф 2 (b): съдържа един-единствен връх с една обратна връзка. Следователно сумарната коригирана сложност е равна на 2.



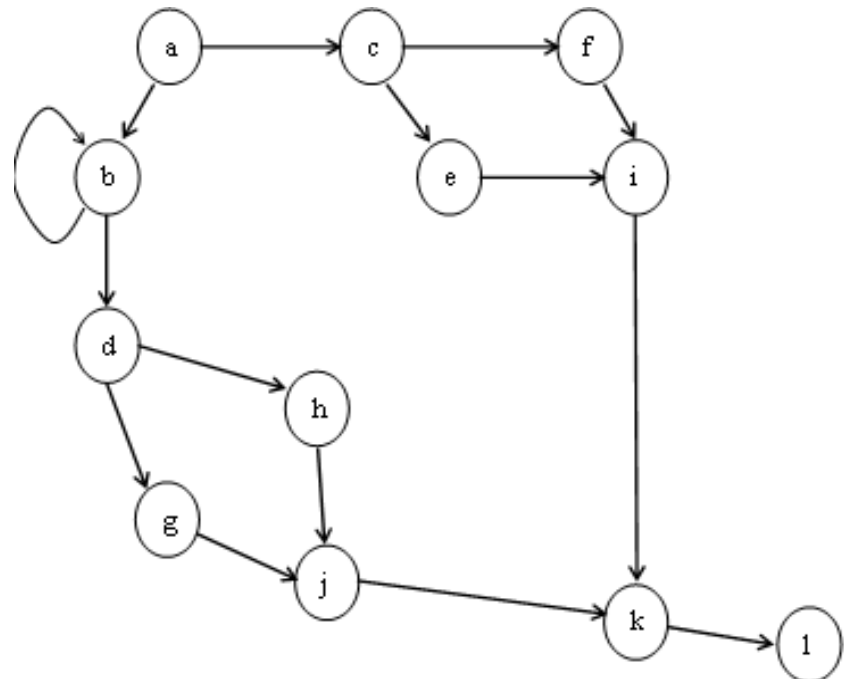
Метрика на граничните стойности - пример

- Коригираната стойност за подграф 3 (c, e, f, i): върхът i е краен. Следва коригираната му сложност да е равна на 0; върховете c, e, f са приемащи върхове, всеки с коригирана сложност 1, следователно подграф 3 има коригирана сложност равна на 3.



Метрика на граничните стойности - пример

- Коригираната стойност за сложността на подграф 4 (d , g , h , j): върхът j е краен, следователно коригираната му сложност е равна на 0; върховете c, e, f са приемащи върхове, всеки с коригираща сложност 1, т.е. подграф 4 има коригирана стойност на сложността, равна на 3.



Метрика на граничните стойности - пример

Връх от графа на програмата	Коригирана сложност на върха
a	10
b	2
c	3
d	3
e	1
f	1
g	1
h	1
i	1
j	1
k	1
l	0
S_a :	25

- Коригираната сложност на върховете на графа на програмата е показана в таблицата в ляво.
- Абсолютната гранична сложност на програмата ще бъде:

$$S_a = 10+2+3+3+1+1+1+1+1+1+1+1+0 = 25$$

- Тъй като общото количество на върховете на графа $V=12$, то

$$S_0 = 1 - \frac{12-1}{25} = 1 - \frac{11}{25} = 0,56$$

- Следователно относителната сложност е равна на 0.56.
- Предимства на метриката: точност и голяма чувствителност към структурата на програмата.

M8. Метрики за сложността на потока от данни

- Тези метрики са описани от Изосимов [6].
- Позволяват да се определи използването, конфигурацията и разместването на данните в програмата. Имаме:
 - **M8.1. Метрика, базирана на обръщенията към глобалните променливи в програмата**
 - **M8.2. Метрика на Чепин за характера на използваните променливи**

М8.1. Метрика, базирана на обръщенията към глобалните променливи в програмата

Нека да означим двойката "модул - глобална променлива" като (p, r) , където:

- p - модул, който има достъп до глобалната променлива r ;
- r - глобална променлива.

В програмата могат да се разглеждат два типа двойки (p, r) :

- **A** - фактически – количеството на реалните обръщения на съществуващите модули към глобалните променливи;
- **P** – възможни - количеството на теоретически възможните обръщения на съществуващите модули към глобалните променливи.

- Тогава сложността на потока от данни може да се определи чрез следното отношение:

$$R = \frac{A}{P}$$

което винаги е по-малко или равно на единица.

- Тази формула показва значимостта на дадена глобална променлива за модула, тоест колкото е по-голяма тази вероятност, толкова е по-голяма е значимостта на променливата за модула.

Метрика, базирана на обръщенията към глобалните променливи в програмата - пример

- Нека в програмата да има 3 глобални променливи и 3 подпрограми. Ако предположим, че всяка подпрограма има достъп до всяка от променливите, то **P = 9**.
- Нека реално първата подпрограма да се обръща към една, втората - към две променливи, а третата към нито една от променливите. Тогава **A = 3**.
- Сложността на потока от данни може да се определи като :

$$R = \frac{3}{9} = 0.33$$

Недостатък: няма достатъчно статистически данни за приложимостта и точността на този метод в практиката.

M8.2. Метрика на Чепин

- Същността на тази метрика се състои в определяне характера на използваните променливи. Те се разбиват на **4 функционални групи** [6].
 1. **P** – променливи за вход, изход и за изчисления.
 2. **M** - модифицируеми или създадени вътре в програмата променливи.
 3. **C** - управляващи променливи, т.е. това са променливи, участващи в управлението на работата на програмния модул.
 4. **T** - “паразитни” променливи, т.е. неизползвани в програмата променливи.
- Възможно е дадена променлива да изпълнява едновременно няколко функции, т.е. необходимо да се отчита участието и във всяка функционална група.

- Метриката на Чепин се определя посредством израза:

$$Q = a1.P + a2.M + a3.C + a4.T ,$$

където **a1**, **a2**, **a3** и **a4** са теглови коефициенти.

- Тези теглови коефициенти се използват за отразяване на различното влияние върху сложността на програмата на всяка функционална група.
- Най - голямо тегло следва да има група **C** (управляващите променливи), тъй като те влияят на потока на управлението на програмата. Авторът на метриката предлага следните конкретни стойности:
- **a1 = 1, a2 = 2, a3 = 3, a4 = 0,5.**

- Тегловният коефициент на “паразитната” група Т не е равен на нула, тъй като “паразитните” променливи не увеличават сложността на потока от данни, но понякога затрудняват нейното разбиране.
- С отчитането на тегловите коефициенти горната формула приема вида :

$$Q = P + 2M + 3C + 0,5T .$$

- С помощта на метриката на Чепин може да се уточни сложността на програма, в която отсъстват неизползвани променливи.
- Може да се направи правомерно изводът, че тази метрика съществено допълва оценката за сложност на програмата.

Забележка:

- Следва да се отбележи, че разгледаните метрики за определяне сложността на програмите са статични и основани на изходните текстове на програмите и графа на програмите, което осигурява **единен подход към автоматизация на тяхното прилагане.**

М9. Метрика за стилистика и разбираемост на програмата

- За времето на своето съществуване програмирането е престанало да бъде само изкуство на самия програмист, а се е превърнало в предмет на колективна производствена дейност.
- Езикът на програмиране и самите програми са не само средство за обръщение на програмиста към компютъра, но и средство за обръщение на програмистите помежду си, т.е. програмирането е интерфейсно средство за общуване между тях [6 - Изосимов, А.В. Метрическая оценка качества программ. М., Издательство МАИ, 1989].

- Възниква необходимост от изискване към **читаемостта (readable) и възприемането на програмите.**
- Читаемостта осигурява:
 - опростяване контактите вътре между разработващите групи;
 - обслужване без участие на производителите;
 - корекции с минимално участие на производителите;
 - достъпност на програмните продукти до широкия кръг потребители.
- Най-простата метрика за стилистика и разбираемост на програмата се явява оценката за **нивото F на коментируемост на програмата:**

$$F = \frac{N_{ком}}{N_{ор}} ,$$

$$F = \frac{N_{ком}}{N_{оп}},$$

- където $N_{ком}$ - количеството на редовете коментари в програмата;
- $N_{оп}$ - общото количество на операторите в изходния текст или броят на редовете в програмата.
- По такъв начин в тази метрика F отразява наситеността на програмата с коментари.
- Практическият опит показва, че $F \geq 0,1$, т.е. средно на всеки 10 реда от програмата се пада по един ред коментари.

Недостатък на **посочената по-горе формула**:

- Както показват изследванията, коментарите се разпределят по текста на програмата неравномерно:
 - **В началото те са много (има излишък)**: описания на операторите, наименования, идентификатори, сведения за изпълнителя, характера и функционалното предназначение на програмата и т.н.
 - **По средата и в края те са по-малко (съществува недостиг)**.
- Така наситената с коментари програма в началото компенсира недостатъка от коментари в тялото на програмата.
- Затова **посочената по-горе формула недостатъчно точно отразява коментируемостта на функционалните части на текста на програмата**.

Преодоляване на недостатъка в посочената по-горе формула за **F**

- По-удобен е вариантът, когато цялата програма се разбива на **n** приблизително равни части и за всяка от тях се определя **F_i** по следната формула:

$$F_i = \left(\frac{N_{комi}}{N_{oni}} - 0,1 \right)$$

- Формира се сумата $F = \sum_{i=1}^n F_i$, която трябва да се сведе до минимум.
 - Ако **F >> 0**, то програмата е пренаситена с коментари.
 - Ако **F << 0**, то коментарите липсват или са твърде малко.
- Нивото на коментируемост на програмата се счита за нормално, ако **F = 0** (на всеки десет реда код има 1 коментарен). В противен случай фрагментите от програмата се допълват с коментари до нормалното ниво на коментируемост.

М10. Метрика за четимост на различни видове документи

I. Описание

Обект на тази метрика [1,2,7] са различни видове документи:

- ръководства за потребителя,
- материали за обучение,
- спецификации и др.

Предлаганата мярка е

- $F = 0.4 * (lm + plw),$

където

- **lm** е средна дължина на изречение, т. е. брой думи/брой изречения;
- **plw** — относителен дял на дългите думи, т. е.

$$plw = \frac{\text{брой дълги думи}}{\text{общ брой думи}} \cdot 100$$

Забележка: В българския език е прието дълги да се наричат думите с над 3 срички.

II. Приложимост

- Мярката на четимост F отразява трудността за четене на текста.

- Боем [8] предлага мярката F да е:

$10 < F < 12$ за обикновените делови документи

и

$12 < F < 16$ за спецификации, документация и научни отчети.

В заключение:

- Съществуват много на брой софтуерни метрики.
- Разработката на метрики е също активна изследователска област.
- Теоретичен и практически интерес представлява изследването на връзките между тях. Също, изключително важен е резултатът от сравняването им.
- Експериментално се твърди, че е достатъчно, към програмата, да се приложат три основни метрики:
 - **метрика за размер,**
 - **метриката на Рехенберг за относителна сложност и**
 - **метриката на Маккейб за структурна сложност,**много други мерки могат да се изведат от резултатите, получени от тези 3 метрики.
- Засега това твърдение е хипотеза, базирана само на експерименти! Би следвало да се докаже и теоретично.

5. Методологични проблеми на използването на софтуерните метрики

- Независимо, че разработката на нови метрики и доказване на валидността на съществуващите е активна изследователска области област, прилагането на метриците, в реални софтуерни проекти е ограничено. Някои от основните причини за това са:
 - а) Участниците в софтуерните разработки **нямат достатъчно добра теоретическа подготовка**, за да оценят полезността на софтуерните метрики.
 - б) Немислимо е използването на метрики в реалните сложни софтуерни проекти, без да са налице **инструментални средства, автоматизиращи процедурите на измерване**.
 - Съществуващите немного такива инструментални средства обикновено са свързани с **точно определена хардуерна и операционна среда**.

- в) **Метриките, измерващи програми, зависят силно и от езика, на който са написани програмите, което прави невъзможно създаването на универсални средства.**
- От прагматична гледна точка интерес представлява следният въпрос: **Как една софтуерна организация, осъзнала полезността на софтуерните метрики, може да изгради и внедри подходяща за нея система от метрики?**

На основата на изследванията в областта на софтуерните метрики и обобщавайки [9], в [1,2] се предлага следната постъпкова процедура:

1. Формулирайте целта, която искате да постигнете с въвеждането на система от метрики (вж. част 3. от класификация А).
2. Изберете свързаните с тази цел софтуерни обекти и съответните им характеристики.

3. Проучете (или възложете проучването на специалист) какви софтуерни метрики съществуват за избраните характеристики.
4. Съставете съвкупност от тези метрики, за които може да се осигурят (чрез закупуване или разработване) инструментални средства, автоматизиращи процедурите на измерване.
5. Регламентируйте и осъществете система за натрупване и анализ на данните от прилагане на метриките и определете контролен срок за експеримента.
6. След изтичане на срока въз основа на експертна оценка на резултатите от прилагане на метриките преустановете експеримента или съставете нова съвкупност от метрики чрез отпадане на някои, добавяне на нови или модифициране на съществуващи. Във втория случай се върнете към стъпка 4.

Въпроси

1. *Какво е измерване в софтуерното производство?*
2. *Как се нарича резултатът от измерването?*
3. *Какво означава „метрика“ в софтуерното производство?*
4. *Каква е разликата между мярка и метрика?*
5. *Начертайте и коментирайте структурния модел на софтуерното измерване.*
6. *С каква цел се прилагат метрики в софтуерното производство?*
7. *Колко типа скали за измерване на стойността на характеристика познавате?*
8. *Избройте някои от класическите софтуерни метрики и опишете такава, по ваш избор.*
9. *Как се класифицират софтуерните метрики, свързани с продукт „програма“ ?*
10. *Опишете метриката на Маккейб за структурна (цикломатична) сложност.*
11. *Коментирайте метрика за оценка на размера на програмата.*
12. *Опишете метрика за стилистика и разбираемост на програмата.*
13. *Защо прилагането на метрики в реални софтуерни проекти е все още ограничено?*

Основна Литература:

1. Нели Манева, Аврам Ескенази, Софтуерни технологии. Анубис София 2001
2. Нели Манева, Аврам Ескенази, Софтуерни технологии. КЛМН, София 2006
3. Sommerville: Software Engineering ,
9. ed. Addison-Wesley, 2011,
4. [http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20\(intro%20txt\)%20-%20I.%20Sommerville%20\(Pearson,%202011\)%20BBS.pdf](http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20(intro%20txt)%20-%20I.%20Sommerville%20(Pearson,%202011)%20BBS.pdf)
5. Fenton N. Software Metrics — A Rigorous Approach. Chapman & Hall, London, 1991
6. Изосимов, А.В. Метрическая оценка качества программ. М., Издательство МАИ, 1989
7. Gunning R. Technique of Clear Writing. New York, Mc Graw Hill, 1968.
8. Boehm et al. Characteristics of Software Quality. TRW Series of Software Technology, vol. 1, North Holland, 1978.
9. Fenton N. Software Metrics — A Rigorous Approach. Chapman & Hall, London, 1991

Допълнителна Литература:

1. Fenton N. Software Metrics — A Rigorous Approach. Chapman & Hall, London, 1991
2. Kitcheman B. Software metrics in Software Reliability Handbook (ed. by P.Rook), London, Elsevier, 1990.
3. Йенсен, Р., Ч.Тониз. Технология на програмирането. Техника, София, 1987.
4. Липаев В. Качество программного обеспечения. М., Финанси и статистика, 1983.
5. McCabe T. A complexity measure. IEEE Transactions on Software Engineering, SE-2,1976, pp. 308—320.
6. Gong, H., M.Schmidt. A complexity Measure Based on Selection and Nesting. ACM SIG-METRICS,Nov.1983.
7. Halstead M.H. Elements of Software Science. New York, Elsevier, 1977.
8. Hocker, H. et al. Comparative Descriptions of Software Quality Measures. GMD-Studien Nr.81,1984.
9. Gordon R.D. Measuring Improvements in Program Clarity. IEEE Trans, on SE, SE-5(2) 1979 pp. 79—90.
10. Budde et al. Untersuchungen ueber Massnahmen zur Verbesserung der Software Production, Teil 1, Bericht Nr. 130 der Gesellschaft fuer Mathematik und Datenverarbeitung. Munchen-Wien, Oldenbourg Verlag, 1980.
11. Rechenberg P. Ein neues Mass fuer die softwaretechnische Komplexitaet von Programmen.Informatik Forsch. Entw. Nr. 1,1986, pp. 26— 37.
12. Gunning R. Technique ofClear Writing. New York, Mc Graw Hill, 1968.
13. Boehm et al. Characteristics ofSoftware Quality. TRW Series ofSoftware Technology, vol. 1, North Holland, 1978.